Contents

1. Objective . . . 3

2. Reference Tools . . . 3

3. Approach . . . 3

4. Architecture . . . 3

4.1 Project files . . . 3

4.2 Deployment architecture . . . 4

4.3 Container image . . . 4

4.4 Manifest design . . . 5

4.5 Running the project . . . 5

4.6 Design notes and constraints . . . 5

5. Source Code . . . 6

5.1 Dockerfile — container image definition . . . 6

5.2 requirements.txt — Python dependencies . . . 6

5.3 train_model.py — model training (runs during the image build) . . . 6

5.4 app.py — inference API (lines 1–60) . . . 7

5.4 app.py — inference API (lines 61–86) . . . 8

5.5 kind-config-task13.yaml — Kind cluster definition . . . 8

5.6 00-namespace.yaml — Namespace . . . 8

5.7 01-configmap.yaml — ConfigMap . . . 8

5.8 02-deployment.yaml — Deployment (lines 1–60) . . . 9

5.8 02-deployment.yaml — Deployment (lines 61–75) . . . 9

5.9 03-service-internal.yaml — ClusterIP Service . . . 10

5.10 04-service-external.yaml — NodePort Service . . . 10

5.11 deploy_dl.py — automation script (lines 1–60) . . . 11

5.11 deploy_dl.py — automation script (lines 61–120) . . . 12

5.11 deploy_dl.py — automation script (lines 121–171) . . . 13

6. Screenshots and Command Output . . . 14

6.1 Container image . . . 14

6.2 Cluster setup . . . 15

6.3 Deployment and services . . . 16

6.4 Verification of pods and services . . . 16

6.5 External accessibility . . . 17

6.6 Load balancing and in-cluster access . . . 18

6.7 Monitoring deployment status . . . 18

7. Results . . . 20

7.1 Deployment summary . . . 20

7.2 Evaluation criteria . . . 21

7.3 Predictions on held-out samples . . . 21

7.4 Load distribution . . . 21

8. Observations . . . 21

9. Known Limitations . . . 22

10. Conclusion . . . 22

1. Objective

To deploy and expose a containerized deep-learning application in a Kubernetes environment: package a trained neural network and its inference API as a Docker image, describe the deployment and the services in YAML manifests, deploy the application to a cluster, expose it outside the cluster and verify that it is accessible.

2. Reference Tools

The deep-learning application is a small neural network (a multi-layer perceptron: 64 inputs → 64 ReLU → 32 ReLU → 10 softmax) trained on the UCI handwritten-digits dataset that ships with scikit-learn (1,797 images of 8×8 pixels). It is wrapped in a **Flask** REST API served by **gunicorn**. **Docker** packages it; **Kind** (Kubernetes IN Docker) provides a local three-node **Kubernetes** cluster; **kubectl** deploys and inspects it. The same procedure applies unchanged to a larger model: only the image content changes.

3. Approach

1. **Create deployment manifests** — Namespace, ConfigMap and a Deployment with 3 replicas, health probes, resource requests/limits, a non-root security context and spread across nodes (Sections 5.6–5.8).
2. **Configure services** — a ClusterIP Service for internal clients and a NodePort Service for external clients (Sections 5.9–5.10).
3. **Deploy applications** — build the image (the model is trained during the build), load it into the Kind nodes and run `kubectl apply -f k8s13/` (Figure 1–Figure 4).
4. **Expose services externally** — Kind maps host port 8090 to NodePort 30090, so the application is available at `http://localhost:8090` (Figure 2, Figure 6).
5. **Verify application accessibility** — health, model-info and prediction requests from outside the cluster, a browser check, load-balancing across pods and a request from inside the cluster through the ClusterIP Service (Figure 7–Figure 10).

The whole procedure is automated by `deploy_dl.py`, which records every command and its real output to `evidence/run_log.json`; the figures and result tables in this report are generated from that log.

4. Architecture

One image, one Deployment, two Services. The model is baked into the image at build time, so each pod is self-contained and any pod can answer any request; the Services provide stable addresses and spread requests over the pods.

4.1 Project files

File	Role
`app/app.py`	Flask API: web page, /health, /model-info, GET /predict/sample/<i> and POST /predict; every response reports the pod that served it.
`app/train_model.py`	Trains the MLP on the digits dataset (80/20 split) and saves the model and metadata; run during the image build.
`app/Dockerfile`	Image definition: python:3.12-slim, dependencies, model training, non-root user, gunicorn entry point.
`app/requirements.txt`	Python dependencies (Flask, gunicorn, NumPy, scikit-learn, joblib).
`kind-config-task13.yaml`	Cluster with one control-plane and two workers; host port 8090 mapped to NodePort 30090.
`k8s13/*.yaml`	Namespace dl-demo, ConfigMap dl-config, Deployment dl-api, Services dl-api-internal and dl-api.
`deploy_dl.py`	Automation and evidence collection: build, cluster, deploy, expose, verify, monitor.
`Task13_DeepLearning_on_Kubernetes.ipynb`	Notebook version of the same workflow (alternative to the .py script).

4.2 Deployment architecture

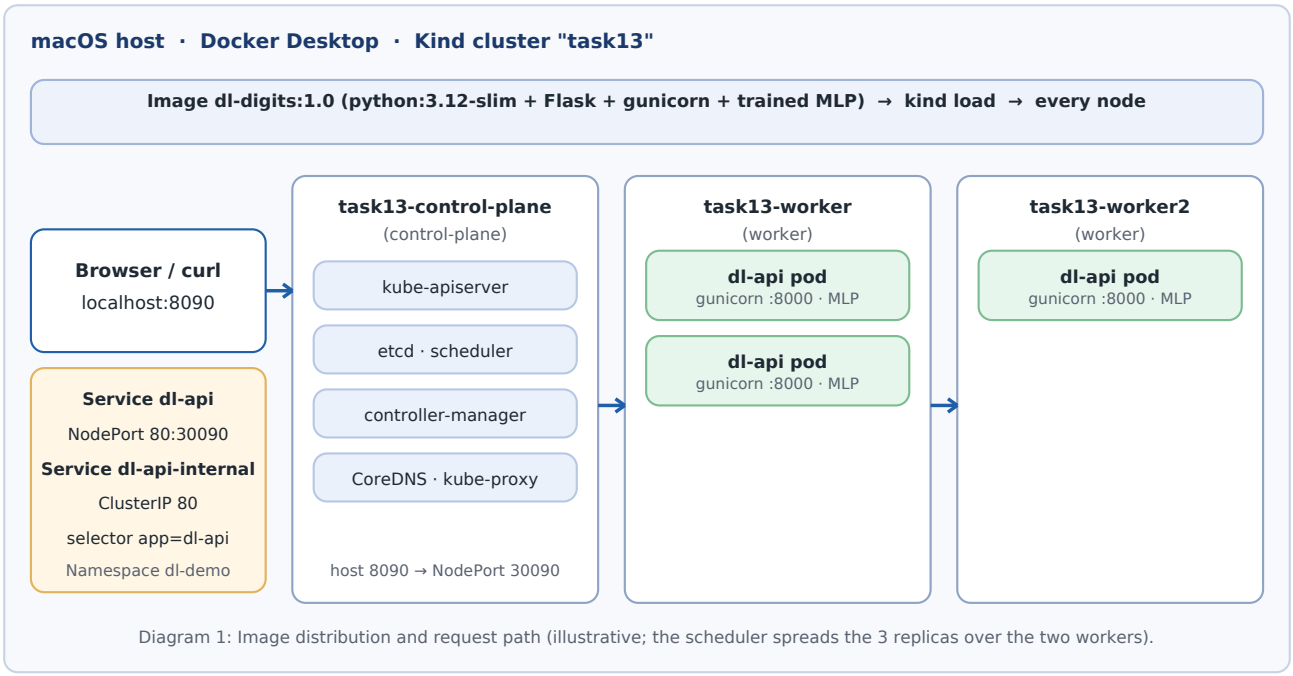


Diagram 1: Image distribution and request path (illustrative; the scheduler spreads the 3 replicas over the two workers).

A request to *localhost:8090* enters the control-plane node through Kind's port mapping, is handled by kube-proxy on NodePort 30090 and is forwarded to one of the ready pods selected by *app=dl-api*. Pods inside the cluster use the ClusterIP Service *dl-api-internal*, which load-balances over the same pods.

4.3 Container image

Aspect	Choice	Reason
Base image	python:3.12-slim	Small Python runtime with a good wheel availability for NumPy and scikit-learn.
Model	Trained in the build (RUN python train_model.py)	Image is self-contained: no external model download or volume at start-up; identical model in every pod.
Server	gunicorn, 2 workers, port 8000	Production-style WSGI server instead of the Flask development server.
User	Non-root UID 10001	Least privilege; enforced again by runAsNonRoot in the Pod spec.

4.4 Manifest design

Setting	Value	Purpose
replicas / strategy	3 · RollingUpdate (maxSurge 1, maxUnavailable 0)	Several pods share the load; updates never reduce capacity.
imagePullPolicy	IfNotPresent	The image exists only locally (loaded into Kind); a registry pull would fail.
envFrom ConfigMap	dl-config (APP_NAME, MODEL_VERSION)	Configuration separated from the image.
env fieldRef	POD_NAME, NODE_NAME	The API reports which pod and node answered, which makes load balancing visible.
probes	startup, readiness and liveness on /health :8000	Traffic only reaches pods whose model is loaded; hung pods are restarted.
resources	requests 100m / 256Mi · limits 500m / 512Mi	Scheduling hint and protection against memory growth.
securityContext	runAsNonRoot, UID 10001, no privilege escalation	Hardening of the container.
topologySpread	maxSkew 1 on hostname (ScheduleAnyway)	Spreads replicas across nodes without blocking scheduling.
Services	ClusterIP 80 → 8000 · NodePort 30090 → 8000	Internal and external access to the same pods.

4.5 Running the project

1. `python deploy_dl.py`
2. `open http://localhost:8090`
3. `python build_report.py # builds the PDF from evidence/`

Prerequisites: Docker Desktop running, `kubectl`, `kind` and Python 3 with Pillow. The script builds the image, creates (or re-uses) the `task13` cluster, loads the image, deploys, verifies and records the evidence; `--cleanup` deletes the cluster afterwards.

4.6 Design notes and constraints

Topic	Detail
Loading the image	Kind nodes cannot see the host's Docker images, so the image is copied in with `kind load docker-image` (the script falls back to `docker save` + `kind load image-archive` if Docker Desktop's image store makes the first method fail).
Port mapping	The host-to-NodePort mapping (8090 → 30090) can only be defined when the cluster is created, so a dedicated cluster configuration is used.
Separate cluster	The cluster is named task13 and uses host port 8090, so it can coexist with a cluster from another task that uses a different port.
Model scope	The model is deliberately small so the image builds quickly; the deployment pattern is independent of model size.

5. Source Code

Every file needed to reproduce the deployment is listed with line numbers: the application and image (5.1–5.4), the cluster and manifests (5.5–5.10) and the automation script (5.11, 171 lines). Lines longer than 118 characters are wrapped and continued with "↳".

5.1 `Dockerfile` — container image definition

```
FROM python:3.12-slim
ENV PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY train_model.py app.py ./
RUN python train_model.py
RUN useradd -u 10001 -m appuser && chown -R appuser /app
USER 10001
EXPOSE 8000
CMD ["gunicorn", "-b", "0.0.0.0:8000", "-w", "2", "--access-logfile", "-", "app:app"]
```

5.2 `requirements.txt` — Python dependencies

```
flask>=3.0
gunicorn>=22.0
numpy>=1.26
scikit-learn>=1.4
joblib>=1.3
```

5.3 `train_model.py` — model training (runs during the image build)

```
"""Train a small neural network (multi-layer perceptron) on the UCI handwritten
digits dataset and save it, together with metadata, to model.joblib.
Runs once at image-build time so every container starts with the model baked in."""
import platform
import joblib
import sklearn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

digits = load_digits()
X, y = digits.data / 16.0, digits.target          # 8x8 images, pixel values 0-16 -> 0-1
idx = range(len(y))
X_tr, X_te, y_tr, y_te, i_tr, i_te = train_test_split(X, y, list(idx), test_size=0.2, stratify=y, random_state=42)

model = MLPClassifier(hidden_layer_sizes=(64, 32), activation="relu", max_iter=600, random_state=42)
model.fit(X_tr, y_tr)
acc = float(model.score(X_te, y_te))

meta = {
    "name": "digits-mlp",
    "task": "handwritten digit classification (0-9)",
    "architecture": "Input 64 -> Dense 64 (ReLU) -> Dense 32 (ReLU) -> Dense 10 (softmax)",
    "parameters": int(sum(w.size for w in model.coefs_) + sum(b.size for b in model.intercepts_)),
    "train_images": len(y_tr),
    "test_images": len(y_te),
    "test_accuracy": round(acc, 4),
    "test_indices": [int(i) for i in i_te],
    "sklearn_version": sklearn.__version__,
    "python_version": platform.python_version(),
}
joblib.dump({"model": model, "meta": meta}, "model.joblib")
print(f"Trained {meta['name']}: {meta['parameters']} parameters, held-out accuracy {acc:.4f}")
```

5.4 `app.py` — inference API (lines 1–60)

```
"""Digit-classifier inference service (Flask) used for the Kubernetes deployment."""
import os
import socket
import time

import joblib
import numpy as np
from flask import Flask, jsonify, render_template_string, request
from sklearn.datasets import load_digits

MODEL_PATH = os.getenv("MODEL_PATH", "model.joblib")
APP_NAME = os.getenv("APP_NAME", "DL Digit Classifier")
MODEL_VERSION = os.getenv("MODEL_VERSION", "1.0")
POD = os.getenv("POD_NAME", socket.gethostname())
NODE = os.getenv("NODE_NAME", "unknown")

bundle = joblib.load(MODEL_PATH)
model, meta = bundle["model"], bundle["meta"]
digits = load_digits()
TEST_IDX = meta["test_indices"]
STARTED = time.time()
app = Flask(__name__)

PAGE = """<!doctype html><title>{{name}}</title>
<body style="font-family:sans-serif;max-width:640px;margin:40px auto;text-align:center">
<h1>{{name}}</h1>
<p>Model <b>{{meta.name}}</b> v{{version}} &middot; held-out accuracy <b>{{acc}}%</b></p>
<p>Served by pod <code>{{pod}}</code> on node <code>{{node}}</code></p>
<button onclick="go()" style="font-size:16px;padding:8px 16px">Classify a random held-out digit</button>
<div id="grid" style="display:grid;grid-template-columns:repeat(8,28px);gap:2px;justify-
↳ content:center;margin:20px"></div>
<h2 id="out"></h2>
<script>
async function go(){
  const i = Math.floor(Math.random()*{{n}});
  const r = await (await fetch('/predict/sample/'+i)).json();
  const g = document.getElementById('grid'); g.innerHTML='';
  r.pixels.forEach(v=>{const d=document.createElement('div');d.style.cssText='width:28px;height:28px;background:rgb('+
↳ (255-v*16)+','+(255-v*16)+','+(255-v*16)+')';g.appendChild(d)});
  document.getElementById('out').textContent = 'Predicted '+r.prediction+' (confidence
↳ '+(r.confidence*100).toFixed(1)+'%) - true label '+r.true_label+(r.correct?' (correct)':' (wrong)')+' - answered by
↳ '+r.pod;
}
go();
</script>"""


def _predict(pixels):
    x = np.asarray(pixels, dtype=float).reshape(1, 64) / 16.0
    p = model.predict_proba(x)[0]
    k = int(np.argmax(p))
    return {"prediction": k, "confidence": round(float(p[k]), 4),
            "probabilities": {str(i): round(float(v), 4) for i, v in enumerate(p)}}


@app.get("/")
def home():
    return render_template_string(PAGE, name=APP_NAME, meta=meta, version=MODEL_VERSION,
                                  acc=round(meta["test_accuracy"] * 100, 2), pod=POD, node=NODE, n=len(TEST_IDX))


@app.get("/health")
def health():
    return jsonify(status="ok", pod=POD, uptime_s=round(time.time() - STARTED, 1))
```

5.4 `app.py` — inference API (lines 61–86)

```
61
62
63  @app.get("/model-info")
64  def model_info():
65      info = {k: v for k, v in meta.items() if k != "test_indices"}
66      return jsonify(app=APP_NAME, version=MODEL_VERSION, pod=POD, node=NODE, **info)
67
68
69  @app.get("/predict/sample/<int:i>")
70  def predict_sample(i):
71      j = TEST_IDX[i % len(TEST_IDX)]                       # always a held-out image
72      out = _predict(digits.data[j])
73      out.update(true_label=int(digits.target[j]), correct=out["prediction"] == int(digits.target[j]),
74                 pixels=[int(v) for v in digits.data[j]], pod=POD)
75      return jsonify(out)
76
77
78  @app.post("/predict")
79  def predict():
80      data = request.get_json(silent=True) or {}
81      px = data.get("pixels")
82      if not isinstance(px, list) or len(px) != 64:
83          return jsonify(error="send JSON {'pixels': [64 numbers between 0 and 16]}"), 400
84      out = _predict(px)
85      out["pod"] = POD
86      return jsonify(out)
```

5.5 `kind-config-task13.yaml` — Kind cluster definition

```
1   kind: Cluster
2   apiVersion: kind.x-k8s.io/v1alpha4
3   nodes:
4     - role: control-plane
5       extraPortMappings:
6         - containerPort: 30090   # NodePort of the dl-api service
7           hostPort: 8090         # http://localhost:8090
8           protocol: TCP
9     - role: worker
10    - role: worker
```

5.6 `00-namespace.yaml` — Namespace

```
1  apiVersion: v1
2  kind: Namespace
3  metadata:
4    name: dl-demo
5    labels:
6      project: task13
```

5.7 `01-configmap.yaml` — ConfigMap

```
1  apiVersion: v1
2  kind: ConfigMap
3  metadata:
4    name: dl-config
5    namespace: dl-demo
6  data:
7    APP_NAME: "DL Digit Classifier on Kubernetes"
8    MODEL_VERSION: "1.0"
```

5.8 `02-deployment.yaml` — Deployment (lines 1–60)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dl-api
  namespace: dl-demo
  labels:
    app: dl-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: dl-api
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: dl-api
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app: dl-api
      containers:
        - name: dl-api
          image: dl-digits:1.0
          imagePullPolicy: IfNotPresent      # image is loaded into Kind, not pulled from a registry
          ports:
            - containerPort: 8000
          envFrom:
            - configMapRef:
                name: dl-config
          env:
            - name: POD_NAME
              valueFrom:
                fieldRef:
                  fieldPath: metadata.name
            - name: NODE_NAME
              valueFrom:
                fieldRef:
                  fieldPath: spec.nodeName
          securityContext:
            allowPrivilegeEscalation: false
          resources:
            requests:
              cpu: 100m
              memory: 256Mi
            limits:
              cpu: 500m
              memory: 512Mi
          startupProbe:
```

5.8 `02-deployment.yaml` — Deployment (lines 61–75)

```yaml
            httpGet:
              path: /health
              port: 8000
            periodSeconds: 3
            failureThreshold: 20
          readinessProbe:
            httpGet:
              path: /health
              port: 8000
            periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /health
              port: 8000
            periodSeconds: 10
```

5.9 `03-service-internal.yaml` — ClusterIP Service

```yaml
apiVersion: v1
kind: Service
metadata:
  name: dl-api-internal
  namespace: dl-demo
spec:
  type: ClusterIP          # reachable only inside the cluster
  selector:
    app: dl-api
  ports:
    - port: 80
      targetPort: 8000
```

5.10 `04-service-external.yaml` — NodePort Service

```yaml
apiVersion: v1
kind: Service
metadata:
  name: dl-api
  namespace: dl-demo
spec:
  type: NodePort           # exposed outside the cluster
  selector:
    app: dl-api
  ports:
    - port: 80
      targetPort: 8000
      nodePort: 30090
```

5.11 `deploy_dl.py` — automation script (lines 1–60)

```
#!/usr/bin/env python3
"""Task 13 - Deploy and expose a containerized deep-learning application on Kubernetes (Kind).

    python deploy_dl.py             # build image, create cluster, deploy, expose, verify
    python deploy_dl.py --cleanup   # also delete the cluster at the end

Every command and its real output is saved to evidence/run_log.json (+ .txt) and rendered as
terminal-style PNG screenshots in evidence/ (needs: pip install pillow).
"""
import argparse, json, os, shutil, subprocess, sys, time, urllib.request, urllib.error
from collections import Counter
from pathlib import Path

ROOT = Path(__file__).parent
EVID = ROOT / "evidence"; EVID.mkdir(exist_ok=True)
NS, CLUSTER, IMAGE = "dl-demo", "task13", "dl-digits:1.0"
BASE = "http://localhost:8090"
LOG = []
BASH = "/bin/bash" if os.path.exists("/bin/bash") else None


def run(title, cmd, check=True, timeout=900):
    print(f"\n=== {title} ===\n$ {cmd}")
    p = subprocess.run(cmd, shell=True, capture_output=True, text=True, timeout=timeout, executable=BASH)
    out = (p.stdout + p.stderr).strip()
    print(out)
    LOG.append({"title": title, "cmd": cmd, "output": out, "rc": p.returncode})
    if check and p.returncode != 0:
        save_logs(); sys.exit(f"Command failed ({p.returncode}): {cmd}")
    return out


def run_live(title, cmd, timeout=3600, keep=22):
    """Run a long command, streaming its output to the terminal; log only the last lines."""
    print(f"\n=== {title} ===\n$ {cmd}")
    p = subprocess.Popen(cmd, shell=True, stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True,
↳ executable=BASH)
    lines, t0 = [], time.time()
    for line in p.stdout:
        print(line, end="", flush=True); lines.append(line.rstrip())
        if time.time() - t0 > timeout:
            p.kill(); break
    p.wait()
    out = "\n".join(lines[-keep:])
    LOG.append({"title": title, "cmd": cmd, "output": out, "rc": p.returncode})
    if p.returncode != 0:
        save_logs(); sys.exit(f"Command failed ({p.returncode}): {cmd}")
    return out


def record(title, cmd, output, rc=0):
    print(f"\n=== {title} ===\n$ {cmd}\n{output}")
    LOG.append({"title": title, "cmd": cmd, "output": output, "rc": rc})


def http(path, data=None, retries=1):
    url = BASE + path
    for k in range(retries):
        try:
            req = urllib.request.Request(url, data=json.dumps(data).encode() if data is not None else None,
                                         headers={"Content-Type": "application/json"} if data is not None else {})
```

5.11 `deploy_dl.py` — automation script (lines 61–120)

```
            return json.loads(urllib.request.urlopen(req, timeout=10).read().decode())
        except Exception as ex:
            err = ex; time.sleep(2)
    raise err


def save_logs():
    (EVID / "run_log.json").write_text(json.dumps(LOG, indent=2))
    with open(EVID / "run_log.txt", "w") as f:
        for e in LOG:
            f.write(f"=== {e['title']} ===\n$ {e['cmd']}\n{e['output']}\n\n")
    try:
        render_screenshots()
    except ImportError:
        print("Pillow not installed - skipping PNG rendering (pip install pillow)")


def render_screenshots():
    from PIL import Image, ImageDraw, ImageFont
    font = None
    for f in ("DejaVuSansMono.ttf", "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf", "Menlo.ttc", "Courier
↳ New.ttf"):
        try:
            font = ImageFont.truetype(f, 15); break
        except OSError:
            continue
    font = font or ImageFont.load_default()
    for i, e in enumerate(LOG, 1):
        lines = [f"$ {e['cmd'][:140]}"] + [l[:140] for l in e["output"].splitlines()[:45]]
        w = max(900, 9 * max(len(l) for l in lines) + 40); h = 40 + 21 * len(lines)
        img = Image.new("RGB", (w, h), (24, 26, 31)); d = ImageDraw.Draw(img)
        d.rectangle([0, 0, w, 26], fill=(50, 54, 62)); d.text((10, 5), e["title"], fill=(220, 220, 220), font=font)
        for n, l in enumerate(lines):
            d.text((14, 36 + 21 * n), l, fill=(120, 220, 140) if n == 0 else (225, 225, 225), font=font)
        img.save(EVID / f"{i:02d}_{e['title'].lower().replace(' ', '_').replace('/', '-')[:40]}.png")


def main():
    ap = argparse.ArgumentParser(); ap.add_argument("--cleanup", action="store_true"); a = ap.parse_args()
    for t in ("docker", "kubectl", "kind"):
        if not shutil.which(t):
            sys.exit(f"'{t}' not found in PATH - install it first (see README.md)")

    # 1. Build the container image (the model is trained inside the build)
    run("Tool versions", "docker --version && kubectl version --client && kind version")
    run_live("Build Docker image", f"docker build --progress=plain --provenance=false --sbom=false -t {IMAGE} app
↳ 2>&1")
    run("Docker image details", "docker image ls dl-digits")

    # 2. Cluster
    existing = run("List kind clusters", "kind get clusters", check=False)
    if CLUSTER not in existing.split():
        run("Create Kind cluster (1 control-plane + 2 workers)",
            f"kind create cluster --name {CLUSTER} --config kind-config-task13.yaml --wait 120s")
    run("Set kubectl context", f"kubectl config use-context kind-{CLUSTER}")
    run("Wait for all nodes Ready", "kubectl wait --for=condition=Ready nodes --all --timeout=180s")
    run("Cluster nodes", "kubectl get nodes -o wide")
    out = run("Load image into Kind", f"kind load docker-image {IMAGE} --name {CLUSTER}", check=False)
    if LOG[-1]["rc"] != 0:      # Docker Desktop's containerd image store can break 'docker-image' loading
        LOG.pop()
        run("Load image into Kind (archive method)",
            f"docker save {IMAGE} -o /tmp/dl-digits.tar && kind load image-archive /tmp/dl-digits.tar --name
↳ {CLUSTER}")
```

5.11 `deploy_dl.py` — automation script (lines 121–171)

```
121
122     # 3. Deploy and expose
123     run("Apply manifests", "kubectl apply -f k8s13/")
124     run("Wait for rollout", f"kubectl rollout status deployment/dl-api -n {NS} --timeout=240s")
125
126     # 4. Verify objects
127     run("Get pods", f"kubectl get pods -n {NS} -o wide")
128     run("Get deployment, services, endpoints", f"kubectl get deploy,svc,endpointslices,cm -n {NS}")
129     run("Describe external service", f"kubectl describe svc dl-api -n {NS}")
130
131     # 5. Verify accessibility from outside the cluster
132     try:
133         h = http("/health", retries=15)
134         record("Health check (external)", f"curl -s {BASE}/health", json.dumps(h, indent=2))
135         record("Model info (external)", f"curl -s {BASE}/model-info", json.dumps(http("/model-info"), indent=2))
136         rows = []
137         for i in range(10):
138             r = http(f"/predict/sample/{i}")
139             rows.append(f"sample {i}: predicted {r['prediction']}  true {r['true_label']}  "
140                         f"confidence {r['confidence']:.3f}  {'OK ' if r['correct'] else 'ERR'}  pod {r['pod']}")
141         record("Predictions on held-out samples", f"curl -s {BASE}/predict/sample/<0..9>", "\n".join(rows))
142         px = http("/predict/sample/0")["pixels"]
143         r = http("/predict", {"pixels": px})
144         top = sorted(r["probabilities"].items(), key=lambda kv: -kv[1])[:3]
145         record("POST /predict with a custom request",
146                f"curl -s -X POST -H 'Content-Type: application/json' -d '{{\"pixels\": {px[:8]}...}}' {BASE}/predict",
147                json.dumps({"prediction": r["prediction"], "confidence": r["confidence"], "top3": dict(top), "pod":
    ↳ r["pod"]}, indent=2))
148         cnt = Counter(http("/health")["pod"] for _ in range(30))
149         record("Load balancing across pods", f"30 x curl -s {BASE}/health",
150                "\n".join(f"{p}: {n} requests" for p, n in sorted(cnt.items())))
151     except Exception as ex:
152         record("HTTP tests", f"curl {BASE}", f"FAILED: {ex}", 1)
153
154     # 6. Verify accessibility from inside the cluster (ClusterIP service)
155     run("In-cluster access via ClusterIP service",
156         f"kubectl run curl-test -n {NS} --rm -i --restart=Never --image=curlimages/curl:8.10.1 -- "
157         f"curl -s http://dl-api-internal/health", check=False, timeout=240)
158
159     # 7. Monitoring
160     run("Pod logs", f"kubectl logs deployment/dl-api -n {NS} --tail=12")
161     run("Deployment details", f"kubectl describe deployment dl-api -n {NS}")
162     run("Recent events", f"kubectl get events -n {NS} --sort-by=.lastTimestamp")
163
164     save_logs()
165     print(f"\nDone. Logs + screenshots saved in {EVID}")
166     if a.cleanup:
167         subprocess.run(f"kind delete cluster --name {CLUSTER}", shell=True)
168
169
170 if __name__ == "__main__":
171     main()
```

6. Screenshots and Command Output

Terminal figures show the exact output captured by `deploy_dl.py` during the run (`evidence/run_log.json`), rendered in a terminal style rather than retyped; the browser figure is a real screen capture. Long outputs are split into parts. Pod names, IPs and timings are those of this particular run.

6.1 Container image

Tool versions

```
$ docker --version && kubectl version --client && kind version
Docker version 29.8.2, build 7fc2dff
Client Version: v1.37.1
Kustomize Version: v5.8.1
kind v0.33.0 go1.27.0 darwin/arm64
```

Build Docker image

```
$ docker build --progress=plain --provenance=false --sbom=false -t dl-digits:1.0 app 2>&1

#8 [4/7] RUN pip install -r requirements.txt
#8 CACHED

#9 [3/7] COPY requirements.txt .
#9 CACHED

#10 [6/7] RUN python train_model.py
#10 CACHED

#11 [7/7] RUN useradd -u 10001 -m appuser && chown -R appuser /app
#11 CACHED

#12 exporting to image
#12 exporting layers done
#12 exporting manifest sha256:3c11f996f9d5b7acd4dc162d58b2e5c7f4bf4b4a28ba507d84d2c9545d0bcf52 done
#12 exporting config sha256:8b10b9689289c69ad3751238d0c80ac21bdc9c2bca322d9d6b28e9eedf7ae397 done
#12 naming to docker.io/library/dl-digits:1.0 done
#12 unpacking to docker.io/library/dl-digits:1.0 done
#12 DONE 0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/zyakllvglgaq629u9i2lozpu7
```

Docker image details

```
$ docker image ls dl-digits
IMAGE           ID             DISK USAGE   CONTENT SIZE   EXTRA
dl-digits:1.0   3c11f996f9d5        603MB          130MB
```

Figure 1: Tool versions, the Docker build of dl-digits:1.0 (the neural network is trained inside the build; last lines shown) and the resulting image.

6.2 Cluster setup

List kind clusters

```
$ kind get clusters
task12
```

Create Kind cluster (1 control-plane + 2 workers)

```
$ kind create cluster --name task13 --config kind-config-task13.yaml --wait 120s
Creating cluster "task13" ...
 • Ensuring node image (kindest/node:v1.37.0) □□  ...
 ✓ Ensuring node image (kindest/node:v1.37.0) □□
 • Preparing nodes □ □ □   ...
 ✓ Preparing nodes □ □ □
 • Writing configuration □  ...
 ✓ Writing configuration □
 • Starting control-plane □□  ...
 ✓ Starting control-plane □□
 • Installing CNI □  ...
 ✓ Installing CNI □
 • Installing StorageClass □  ...
 ✓ Installing StorageClass □
 • Joining worker nodes □  ...
 ✓ Joining worker nodes □
 • Waiting ≤ 2m0s for control-plane = Ready □  ...
 ✓ Waiting ≤ 2m0s for control-plane = Ready □
 • Ready after 6s □
Set kubectl context to "kind-task13"
You can now use your cluster with:

kubectl cluster-info --context kind-task13

Thanks for using kind! □
```

Set kubectl context

```
$ kubectl config use-context kind-task13
Switched to context "kind-task13".
```

Figure 2: Creation of the three-node Kind cluster task13 (host port 8090 mapped to NodePort 30090) and selection of its kubectl context.

Wait for all nodes Ready

```
$ kubectl wait --for=condition=Ready nodes --all --timeout=180s
node/task13-control-plane condition met
node/task13-worker condition met
node/task13-worker2 condition met
```

Cluster nodes

```
$ kubectl get nodes -o wide
NAME                  STATUS   ROLES           AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE                       KERNEL-VERSION
    CONTAINER-RUNTIME
task13-control-plane   Ready    control-plane   28s   v1.37.0   172.18.0.5    <none>        Debian GNU/Linux 13 (trixie)   7.0.14-linuxkit (arm64)
    containerd://2.3.4
task13-worker          Ready    <none>          12s   v1.37.0   172.18.0.7    <none>        Debian GNU/Linux 13 (trixie)   7.0.14-linuxkit (arm64)
    containerd://2.3.4
task13-worker2         Ready    <none>          13s   v1.37.0   172.18.0.6    <none>        Debian GNU/Linux 13 (trixie)   7.0.14-linuxkit (arm64)
    containerd://2.3.4
```

Load image into Kind

```
$ kind load docker-image dl-digits:1.0 --name task13
Image: "dl-digits:1.0" with ID "sha256:3c11f996f9d5b7acd4dc162d58b2e5c7f4bf4b4a28ba507d84d2c9545d0bcf52" not yet present on node "task13-control-
    plane", loading...
Image: "dl-digits:1.0" with ID "sha256:3c11f996f9d5b7acd4dc162d58b2e5c7f4bf4b4a28ba507d84d2c9545d0bcf52" not yet present on node "task13-worker",
    loading...
Image: "dl-digits:1.0" with ID "sha256:3c11f996f9d5b7acd4dc162d58b2e5c7f4bf4b4a28ba507d84d2c9545d0bcf52" not yet present on node "task13-worker2",
    loading...
```

Figure 3: All nodes Ready, node listing, and the image loaded into every Kind node (Kind cannot pull a locally built image from a registry).

6.3 Deployment and services

Apply manifests

```
$ kubectl apply -f k8s13/
namespace/dl-demo created
configmap/dl-config created
deployment.apps/dl-api created
service/dl-api-internal created
service/dl-api created
```

Wait for rollout

```
$ kubectl rollout status deployment/dl-api -n dl-demo --timeout=240s
Waiting for deployment "dl-api" rollout to finish: 0 of 3 updated replicas are available...
Waiting for deployment "dl-api" rollout to finish: 1 of 3 updated replicas are available...
Waiting for deployment "dl-api" rollout to finish: 2 of 3 updated replicas are available...
deployment "dl-api" successfully rolled out
```

Figure 4: kubectl apply -f k8s13/ creates the Namespace, ConfigMap, Deployment and both Services; rollout status waits for all replicas.

6.4 Verification of pods and services

Get pods

```
$ kubectl get pods -n dl-demo -o wide
NAME                      READY   STATUS    RESTARTS   AGE   IP           NODE            NOMINATED NODE   READINESS GATES
dl-api-794dbc8454-p8kt7   1/1     Running   0          6s    10.244.2.2   task13-worker   <none>           <none>
dl-api-794dbc8454-rkgvv   1/1     Running   0          6s    10.244.2.3   task13-worker   <none>           <none>
dl-api-794dbc8454-stzkv   1/1     Running   0          6s    10.244.1.2   task13-worker2  <none>           <none>
```

Get deployment, services, endpoints

```
$ kubectl get deploy,svc,endpointslices,cm -n dl-demo
NAME                     READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/dl-api   3/3     3            3           7s

NAME                     TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
service/dl-api           NodePort    10.96.2.238    <none>        80:30090/TCP   7s
service/dl-api-internal  ClusterIP   10.96.255.21   <none>        80/TCP         7s

NAME                                                ADDRESSTYPE   PORTS   ENDPOINTS                         AGE
endpointslice.discovery.k8s.io/dl-api-5npt9         IPv4          8000    10.244.1.2,10.244.2.3,10.244.2.2  7s
endpointslice.discovery.k8s.io/dl-api-internal-k2zxk  IPv4          8000    10.244.1.2,10.244.2.3,10.244.2.2  7s

NAME                        DATA   AGE
configmap/dl-config         2      7s
configmap/kube-root-ca.crt  1      7s
```

Figure 5: Pods with their IPs and nodes, and the Deployment, Services (ClusterIP and NodePort), EndpointSlices and ConfigMaps.

Describe external service

```
$ kubectl describe svc dl-api -n dl-demo
Name:                     dl-api
Namespace:                dl-demo
Labels:                   <none>
Annotations:              <none>
Selector:                 app=dl-api
Type:                     NodePort
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.96.2.238
IPs:                      10.96.2.238
Port:                     <unset>  80/TCP
TargetPort:               8000/TCP
NodePort:                 <unset>  30090/TCP
Endpoints:                10.244.1.2:8000,10.244.2.3:8000,10.244.2.2:8000
Session Affinity:         None
External Traffic Policy:  Cluster
Internal Traffic Policy:  Cluster
Events:                   <none>
```

Figure 6: kubectl describe svc dl-api: NodePort 30090 → targetPort 8000, selector app=dl-api and the pod endpoints.

6.5 External accessibility

Health check (external)

```
$ curl -s http://localhost:8090/health
{
  "pod": "dl-api-794dbc8454-stzkv",
  "status": "ok",
  "uptime_s": 1.5
}
```

Model info (external)

```
$ curl -s http://localhost:8090/model-info
{
  "app": "DL Digit Classifier on Kubernetes",
  "architecture": "Input 64 -> Dense 64 (ReLU) -> Dense 32 (ReLU) -> Dense 10 (softmax)",
  "name": "digits-mlp",
  "node": "task13-worker2",
  "parameters": 6570,
  "pod": "dl-api-794dbc8454-stzkv",
  "python_version": "3.12.15",
  "sklearn_version": "1.9.1",
  "task": "handwritten digit classification (0-9)",
  "test_accuracy": 0.9778,
  "test_images": 360,
  "train_images": 1437,
  "version": "1.0"
}
```

Figure 7: Health check and model metadata requested from outside the cluster at http://localhost:8090.

Predictions on held-out samples

```
$ curl -s http://localhost:8090/predict/sample/<0..9>
sample 0: predicted 5  true 5  confidence 1.000  OK   pod dl-api-794dbc8454-stzkv
sample 1: predicted 2  true 2  confidence 1.000  OK   pod dl-api-794dbc8454-stzkv
sample 2: predicted 8  true 8  confidence 0.997  OK   pod dl-api-794dbc8454-stzkv
sample 3: predicted 1  true 1  confidence 0.837  OK   pod dl-api-794dbc8454-stzkv
sample 4: predicted 7  true 7  confidence 1.000  OK   pod dl-api-794dbc8454-stzkv
sample 5: predicted 2  true 2  confidence 0.991  OK   pod dl-api-794dbc8454-stzkv
sample 6: predicted 6  true 6  confidence 1.000  OK   pod dl-api-794dbc8454-stzkv
sample 7: predicted 2  true 2  confidence 1.000  OK   pod dl-api-794dbc8454-stzkv
sample 8: predicted 6  true 6  confidence 1.000  OK   pod dl-api-794dbc8454-stzkv
sample 9: predicted 5  true 5  confidence 1.000  OK   pod dl-api-794dbc8454-stzkv
```

POST /predict with a custom request

```
$ curl -s -X POST -H 'Content-Type: application/json' -d '{"pixels": [0, 0, 8, 12, 14, 12, 3, 0]...}' http://localhost:8090/predict
{
  "prediction": 5,
  "confidence": 0.9996,
  "top3": {
    "5": 0.9996,
    "9": 0.0004,
    "0": 0.0
  },
  "pod": "dl-api-794dbc8454-stzkv"
}
```

Figure 8: Predictions on ten held-out digit images (GET) and a custom POST /predict request; every response names the pod that answered.

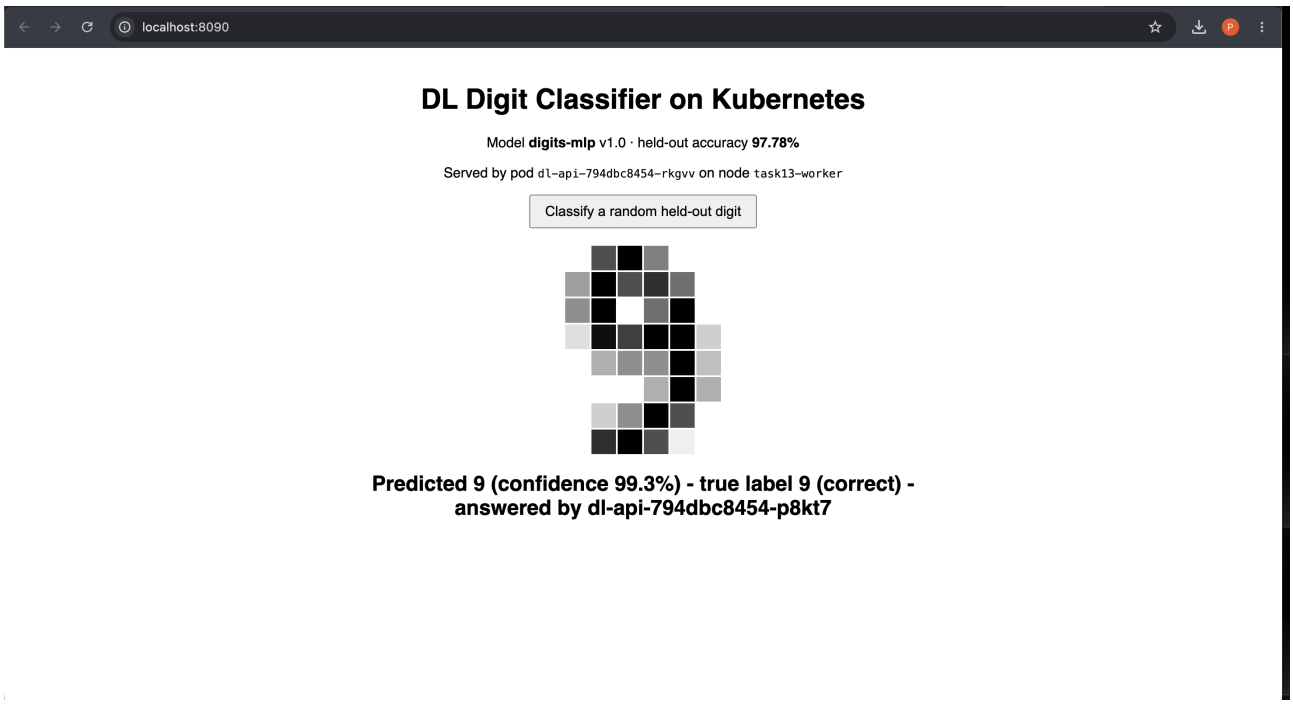


Figure 9: The application opened in a browser at http://localhost:8090 (classifying a random held-out digit).

6.6 Load balancing and in-cluster access

Load balancing across pods

```
$ 30 x curl -s http://localhost:8090/health
dl-api-794dbc8454-stzkv: 30 requests
```

In-cluster access via ClusterIP service

```
$ kubectl run curl-test -n dl-demo --rm -i --restart=Never --image=curlimages/curl:8.10.1 -- curl -s http://dl-api-internal/health
{"pod":"dl-api-794dbc8454-rkgvv","status":"ok","uptime_s":9.8}
pod "curl-test" deleted from dl-demo namespace
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed through the
    command prompt.
If you don't see a command prompt, try pressing enter.
```

Figure 10: Thirty requests to the NodePort Service are spread across the pods; the ClusterIP Service is reached from a temporary pod inside the cluster.

6.7 Monitoring deployment status

Pod logs

```
$ kubectl logs deployment/dl-api -n dl-demo --tail=12
[2026-10-09 16:07:33 +0000] [1] [INFO] Starting gunicorn 26.2.0
[2026-10-09 16:07:33 +0000] [1] [INFO] Listening at: http://0.0.0.0:8000 (1)
[2026-10-09 16:07:33 +0000] [1] [INFO] Using worker: sync
[2026-10-09 16:07:33 +0000] [11] [INFO] Booting worker with pid: 11
[2026-10-09 16:07:33 +0000] [12] [INFO] Booting worker with pid: 12
[2026-10-09 16:07:33 +0000] [1] [INFO] Control socket listening at /home/appuser/.gunicorn/gunicorn.ctl
10.244.2.1 - - [09/Oct/2026:16:07:37 +0000] "GET /health HTTP/1.1" 200 63 "-" "kube-probe/1.37"
10.244.2.1 - - [09/Oct/2026:16:07:38 +0000] "GET /health HTTP/1.1" 200 63 "-" "kube-probe/1.37"
10.244.2.1 - - [09/Oct/2026:16:07:38 +0000] "GET /health HTTP/1.1" 200 63 "-" "kube-probe/1.37"
10.244.2.1 - - [09/Oct/2026:16:07:42 +0000] "GET /health HTTP/1.1" 200 63 "-" "kube-probe/1.37"
10.244.2.1 - - [09/Oct/2026:16:07:43 +0000] "GET /health HTTP/1.1" 200 63 "-" "kube-probe/1.37"
10.244.2.1 - - [09/Oct/2026:16:07:48 +0000] "GET /health HTTP/1.1" 200 64 "-" "kube-probe/1.37"
Found 3 pods, using pod/dl-api-794dbc8454-p8kt7
```

Figure 11: Application logs of one pod (gunicorn access log, including probe requests).

Deployment details

```
$ kubectl describe deployment dl-api -n dl-demo
Name:                   dl-api
Namespace:              dl-demo
CreationTimestamp:      Fri, 09 Oct 2026 21:37:32 +0530
Labels:                 app=dl-api
Annotations:            deployment.kubernetes.io/revision: 1
Selector:               app=dl-api
Replicas:               3 desired | 3 updated | 3 total | 3 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  0 max unavailable, 1 max surge
Pod Template:
  Labels:  app=dl-api
  Containers:
   dl-api:
    Image:      dl-digits:1.0
    Port:       8000/TCP
    Host Port:  0/TCP
    Limits:
      cpu:     500m
      memory:  512Mi
    Requests:
      cpu:      100m
      memory:   256Mi
    Liveness:   http-get http://:8000/health delay=0s timeout=1s period=10s successThreshold=1 failureThreshold=3
```

Figure 12 (part 1 of 2): kubectl describe deployment dl-api: replicas, rolling-update strategy, image, ports, probes, resources, environment from the ConfigMap and the pod fields, and the ReplicaSet.

Deployment details

```
$ kubectl describe deployment dl-api -n dl-demo
    Readiness:  http-get http://:8000/health delay=0s timeout=1s period=5s successThreshold=1 failureThreshold=3
    Startup:    http-get http://:8000/health delay=0s timeout=1s period=3s successThreshold=1 failureThreshold=20
    Environment Variables from:
      dl-config  ConfigMap  Optional: false
    Environment:
      POD_NAME:                 (v1:metadata.name)
      NODE_NAME:                (v1:spec.nodeName)
    Mounts:                     <none>
  Volumes:                      <none>
  Topology Spread Constraints:  kubernetes.io/hostname:ScheduleAnyway when max skew 1 is exceeded for selector app=dl-api
  Node-Selectors:               <none>
  Tolerations:                  <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets:  <none>
NewReplicaSet:   dl-api-794dbc8454 (3/3 replicas created)
Events:
  Type    Reason             Age   From                   Message
  ----    ------             ----  ----                   -------
  Normal  ScalingReplicaSet  17s   deployment-controller  Scaled up replica set dl-api-794dbc8454 from 0 to 3
```

Figure 12 (part 2 of 2): continued.

Recent events

```
$ kubectl get events -n dl-demo --sort-by=.lastTimestamp
LAST SEEN   TYPE      REASON              OBJECT                          MESSAGE
17s         Normal    Created             pod/dl-api-794dbc8454-rkgvv     Container created
17s         Normal    Pulled              pod/dl-api-794dbc8454-p8kt7     Container image "dl-digits:1.0" already present on machine and can be
    accessed by the pod
17s         Normal    ScalingReplicaSet   deployment/dl-api               Scaled up replica set dl-api-794dbc8454 from 0 to 3
17s         Normal    SuccessfulCreate    replicaset/dl-api-794dbc8454    Created pod: dl-api-794dbc8454-rkgvv
17s         Normal    SuccessfulCreate    replicaset/dl-api-794dbc8454    Created pod: dl-api-794dbc8454-p8kt7
17s         Normal    Scheduled           pod/dl-api-794dbc8454-p8kt7     Successfully assigned dl-demo/dl-api-794dbc8454-p8kt7 to task13-worker
17s         Normal    Pulled              pod/dl-api-794dbc8454-rkgvv     Container image "dl-digits:1.0" already present on machine and can be
    accessed by the pod
17s         Normal    Created             pod/dl-api-794dbc8454-p8kt7     Container created
17s         Normal    SuccessfulCreate    replicaset/dl-api-794dbc8454    Created pod: dl-api-794dbc8454-stzkv
17s         Normal    Created             pod/dl-api-794dbc8454-stzkv     Container created
17s         Normal    Scheduled           pod/dl-api-794dbc8454-stzkv     Successfully assigned dl-demo/dl-api-794dbc8454-stzkv to task13-worker2
17s         Normal    Scheduled           pod/dl-api-794dbc8454-rkgvv     Successfully assigned dl-demo/dl-api-794dbc8454-rkgvv to task13-worker
17s         Normal    Pulled              pod/dl-api-794dbc8454-stzkv     Container image "dl-digits:1.0" already present on machine and can be
    accessed by the pod
16s         Normal    Started             pod/dl-api-794dbc8454-rkgvv     Container started
16s         Normal    Started             pod/dl-api-794dbc8454-stzkv     Container started
16s         Normal    Started             pod/dl-api-794dbc8454-p8kt7     Container started
13s         Warning   Unhealthy           pod/dl-api-794dbc8454-rkgvv     Startup probe failed: Get "http://10.244.2.3:8000/health": context deadline
    exceeded (Client.Timeout exceeded while awaiting headers)
13s         Warning   Unhealthy           pod/dl-api-794dbc8454-p8kt7     Startup probe failed: Get "http://10.244.2.2:8000/health": context deadline
    exceeded (Client.Timeout exceeded while awaiting headers)
13s         Warning   Unhealthy           pod/dl-api-794dbc8454-stzkv     Startup probe failed: Get "http://10.244.1.2:8000/health": context deadline
    exceeded (Client.Timeout exceeded while awaiting headers)
10s         Normal    Pulling             pod/curl-test                   Pulling image "curlimages/curl:8.10.1"
10s         Normal    Scheduled           pod/curl-test                   Successfully assigned dl-demo/curl-test to task13-worker2
2s          Normal    Started             pod/curl-test                   Container started
2s          Normal    Created             pod/curl-test                   Container created
2s          Normal    Pulled              pod/curl-test                   Successfully pulled image "curlimages/curl:8.10.1" in 7.496s (7.496s
    including waiting). Image size: 9917437 bytes.
```

Figure 13: Namespace events sorted by time: scheduling, container start and probe activity.

7. Results

The values below are taken from the run log; the figures referenced show the original output.

7.1 Deployment summary

Item	Result	Evidence
Image	dl-digits:1.0 built from python:3.12-slim; MLP with 6,570 parameters trained during the build	Figure 1
Cluster	task13: 3 nodes — task13-control-plane (Ready), task13-worker (Ready), task13-worker2 (Ready)	Figure 3
Objects	Namespace dl-demo, ConfigMap dl-config, Deployment dl-api, Services dl-api-internal (ClusterIP) and dl-api (NodePort 30090)	Figure 4
Pods	3 of 3 pods Running 1/1 (placement shown in the figure)	Figure 5
Health check	GET /health through localhost:8090 → status "ok" from pod dl-api-794dbc8454-stzkv	Figure 7
Model	digits-mlp: Input 64 -> Dense 64 (ReLU) -> Dense 32 (ReLU) -> Dense 10 (softmax); held-out accuracy 97.78% on 360 test images	Figure 7
Predictions	10 of 10 held-out samples classified correctly through the Service	Figure 8
Load balancing	30 requests answered by 1 distinct pods: stzkv: 30	Figure 10
Internal access	ClusterIP Service dl-api-internal reachable from inside the cluster	Figure 10

7.2 Evaluation criteria

Criterion	How it was met	Status
Successful deployment	Image built and loaded; Deployment rolled out; 3 of 3 pods Running and ready.	Met
Service accessibility	API reachable through the NodePort from the host (curl and browser) and through the ClusterIP Service from inside the cluster; requests are spread over the pods.	Met
Configuration quality	Declarative manifests with ConfigMap-based configuration, three probes, resource requests/limits, non-root security context, rolling-update strategy and topology spread; the cluster and image are reproducible from the listed files.	Met

7.3 Predictions on held-out samples

Sample	Predicted	True	Confidence	Result	Answered by pod
0	5	5	1.000	correct	dl-api-794dbc8454-stzkv
1	2	2	1.000	correct	dl-api-794dbc8454-stzkv
2	8	8	0.997	correct	dl-api-794dbc8454-stzkv
3	1	1	0.837	correct	dl-api-794dbc8454-stzkv
4	7	7	1.000	correct	dl-api-794dbc8454-stzkv
5	2	2	0.991	correct	dl-api-794dbc8454-stzkv
6	6	6	1.000	correct	dl-api-794dbc8454-stzkv
7	2	2	1.000	correct	dl-api-794dbc8454-stzkv
8	6	6	1.000	correct	dl-api-794dbc8454-stzkv
9	5	5	1.000	correct	dl-api-794dbc8454-stzkv

The samples come from the 20% test split that the model never saw during training; the ten requests show per-image behaviour, while the accuracy of 97.78% is measured on all 360 test images at training time.

7.4 Load distribution

Pod	Requests (of 30)
dl-api-794dbc8454-stzkv	30

8. Observations

Self-contained image. Training the model during the build makes every pod identical and removes any runtime dependency on a model store; the price is that a new model needs a new image and a rollout.

Image distribution in Kind. The image exists only in the local Docker daemon, so it must be loaded into every Kind node and the Deployment must use `imagePullPolicy: IfNotPresent`; with a registry, the same manifest would simply reference the registry image.

Probes protect the model start-up. The startup probe gives the pod time to import scikit-learn and load the model; readiness keeps it out of the Service until it answers, and liveness restarts it if it hangs.

Two Services, one set of pods. `dl-api-internal` (ClusterIP) serves clients inside the cluster, `dl-api` (NodePort 30090) serves clients outside; both select `app=dl-api` and therefore always target exactly the ready pods.

Load balancing is visible. Because each response contains the pod name, distribution can be observed directly. The 30 requests were answered by 1 different pods (30 requests respectively), showing that the

Service distributes traffic across replicas.

Configuration is externalised. Application name and model version come from the ConfigMap and pod identity from the downward API, so the same image runs unchanged in any environment.

Hardening. The container runs as a non-root user with privilege escalation disabled and has CPU and memory limits, which are good defaults for any model-serving workload.

9. Known Limitations

• **Small model.** An MLP on 8×8 digits demonstrates the deployment pattern, not production-scale deep learning; larger models need more memory, possibly GPU nodes and a registry.

• **Local cluster.** All nodes are containers on one machine, so real network paths, node failures and cloud load balancers are not exercised.

• **NodePort exposure.** External access relies on Kind's host port mapping; production clusters would normally use an Ingress or a LoadBalancer Service with TLS.

• **No autoscaling or monitoring stack.** There is no metrics-server, HorizontalPodAutoscaler or Prometheus; the replica count is fixed at 3.

• **No model versioning.** The model is part of the image, and there is no A/B or canary rollout between model versions.

10. Conclusion

A containerized neural-network classifier was packaged with Docker, loaded into a three-node Kind cluster and deployed through declarative manifests with configuration, probes, resource limits and a hardened security context. It was exposed externally through a NodePort Service and internally through a ClusterIP Service, and its accessibility was verified with health, model-info and prediction requests, a browser check and a load-distribution test. The deliverables are the Kubernetes deployment files (Section 5), the service configurations (5.9–5.10), the screenshots and outputs (Section 6) and this deployment report.